lecture_02.py

```python
import math
import torch.nn.functional as F
import timeit
from typing import Iterable
import torch
from torch import nn
from einops import rearrange, einsum, reduce

from edtrace import text, image, link
from lecture_util import article_link
from gpu_util import cuda_if_available, get_max_memory_usage
from facts import h100_flop_per_sec, h100_bytes_per_sec
from references import deepseek_v3_2_2025, adagrad_2011, nemotron_3_super_2026


def main():
    text("""
    Announcements:
    - Join the CS336 slack
    - Sign up on Modal with your **Stanford** email
    - Read the AI policy guide
    - Read the cluster guide

    Marin 1e23 FLOPs run finished and matched forecasts!
    """)
    image("https://cs336.stanford.edu/lectures/images/delphi-scaling-suite-results.png", box=(0.123, 0.469, 0.877, 0.778))
    text("""
    Last lecture: overview, tokenization
    Today: resource accounting (systems)

    Recall: what's the best model one can train given fixed resources (compute, memory)?
    In other words: maximize (computational) **efficiency**.
    Prerequisite: understand the resources (compute, memory) for a given computation.
    """)
    motivating_questions()


```

What knowledge to take away from this lecture:

- Mechanics: straightforward (PyTorch semantics)
- Mindset: resource accounting (remember to do it)
- Intuitions: get a sense of how resources are spent, no ML magic today

```
# Memory accounting
tensors_basics()
tensors_memory()
tensors_on_gpus()

# Compute accounting
tensor_einops()
tensor_operations_flops()

arithmetic_intensity()

# Memory and compute accounting for training
deep_network()
gradients_basics()
gradients_flops()
optimizer()
train_loop()

# More memory optimizations
gradient_accumulation()
activation_checkpointing()
```

Summary:

- Everything is operations on tensors (parameters, gradients, activations, optimizer states, data)
- einops: better way to think about tensor operations
- 6 (# data points) (# parameters) FLOPs per training step
- Arithmetic intensity / roofline analysis: compute-bound or memory-bound?
- Matrix multiplications are compute-bound, elementwise operations are memory-bound
- Gradient accumulation, activation checkpointing: reduce memory to use bigger batch sizes

```
def motivating_questions():
```

Question: How long would it take to train a 70B parameter model on 15T tokens on 1024 H100s?

```
total_flops = 6 * 70e9 * 15e12
h100_flop_per_sec = 1979e12 / 2
mfu = 0.5
flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
days = total_flops / flops_per_day
```

Question: What's the largest model that can you can train on 8 H100s using AdamW?

```
h100_bytes = 80e9
bytes_per_parameter = 2 + 2 + (4 + 4)  # parameters (2), gradients (2), optimizer state (4 + 4)
num_parameters = (h100_bytes * 8) / bytes_per_parameter
```

Caveat: activations are not accounted for (depends on batch size and sequence length), so this is an upper bound.

This is a rough back-of-the-envelope calculation.
But it gives you the flavor of napkin math one can quickly do to get a sense of resources.

```
def tensors_basics():
```

Tensors are the basic building block for storing everything:

- data

- parameters
- gradients
- optimizer state
- activations

Example: parameters of the DeepSeek v3.2 model [DeepSeek-AI+ 2025]
[DeepSeek v3.2 model on Hugging Face]

Each tensor has a rank, which is the number of dimensions.

```
x = torch.zeros(4)        # rank 1 tensor (vector)
x = torch.zeros(4, 8)     # rank 2 tensor (matrix)
x = torch.zeros(4, 8, 2)  # rank 3 tensor
```

In Transformers, will see tensors of rank 4:

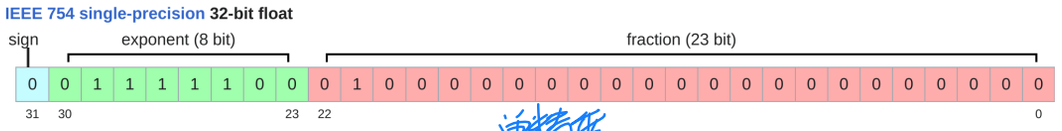
```
B = 32   # Batch size
S = 16   # Sequence length
H = 16   # Number of heads
D = 64   # Hidden dimension per head
x = torch.zeros(B, S, H, D)
```

```
def tensors_memory():
```

Elements of tensors are generally floating point numbers.

fp32

[Wikipedia]



The fp32 data type (also known as float32 or single precision) is the default.
Traditionally, in scientific computing, fp32 is the baseline; you could use double precision (fp64) in some cases.
In deep learning, you can be a lot sloppier.

Let's examine memory usage of these tensors.
Memory is determined by the (i) number of values and (ii) data type of each value.

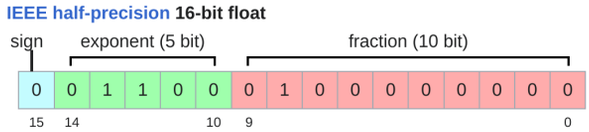
```
x = torch.zeros(4, 8)
assert x.dtype == torch.float32  # Default type
assert x.numel() == 4 * 8
assert x.element_size() == 4  # Float is 4 bytes
assert get_memory_usage(x) == 4 * 8 * 4  # 128 bytes
```

One matrix in the feedforward layer of GPT-3:

```
assert get_memory_usage(torch.empty(12288 * 4, 12288)) == 2304 * 1024 * 1024  # 2.3 GB
```

fp16

[Wikipedia]



The fp16 data type (also known as float16 or half precision) cuts down the memory.

```
x = torch.zeros(4, 8, dtype=torch.float16)
assert x.element_size() == 2
```

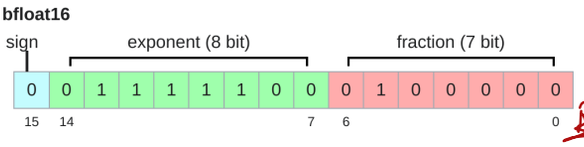
However, the dynamic range (especially for small numbers) isn't great.

```
x = torch.tensor([1e-8], dtype=torch.float16)
assert x == 0  # Underflow!
```

If this happens when you train, you can get instability.

bf16

[Wikipedia]



Google Brain developed brain floating point (bf16) in 2018 to address this issue.
bf16 uses the same memory as fp16 but has the same dynamic range as fp32!
The only catch is that the resolution is worse, but this matters less for deep learning.

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
assert x != 0  # No underflow!
```

Mixed precision

Implications on training:

- Training with fp32 works, but requires lots of memory.
- Training with fp16 and even bf16 is risky, and you can get instability.

Solution: mixed precision training [Micikevicius+ 2017]

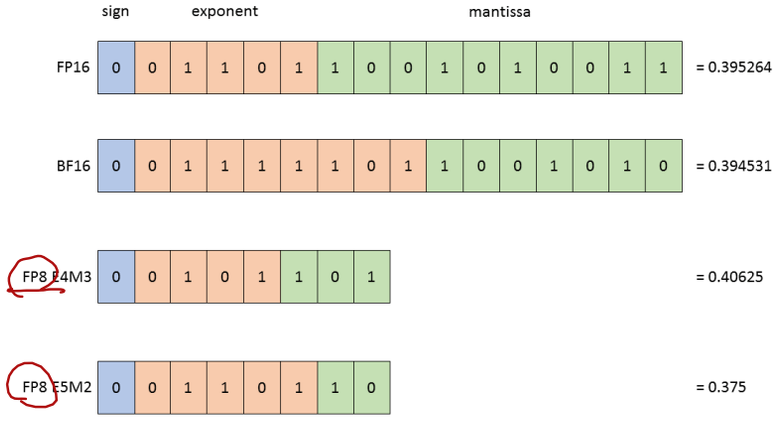
- Use bf16 for parameters, activations, and gradients
- Use fp32 for optimizer states

Pytorch has an automatic mixed precision (AMP) library. [docs]
Tries to cast things into bf16 when safe (matmuls, not exp).

```
with torch.amp.autocast("cuda", dtype=torch.bfloat16):
    x = torch.zeros(4, 8)
```

fp8

In 2022, fp8 was standardized, motivated by machine learning workloads primer.



H100s support two variants of FP8: E4M3 (range [-448, 448]) and E5M2 ([-57344, 57344]).
Reference: [Micikevicius+ 2022]

fp4

In 2025, NVIDIA developed nvfp4
Only 4 bits per value!
Values: -6, -4, -3, -2, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2, 3, 4, 6
Use a separate scale factor per block, so actually get more dynamic range (but just can't vary freely from neighbors).
Nemotron 3 Super was trained in NVFP4
[Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning]

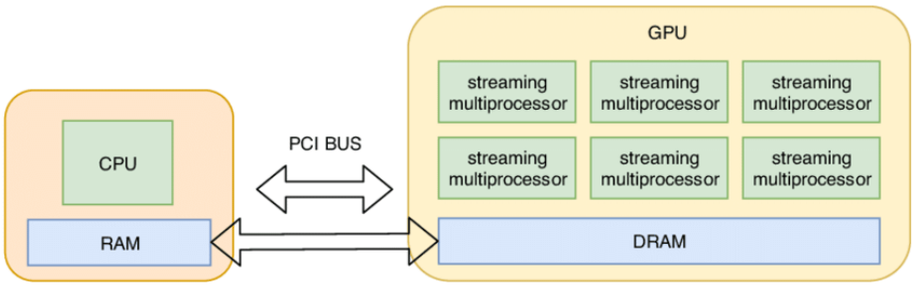
Some of this is done in NVIDIA libraries outside of user control.

```
def tensors_on_gpus():
```

By default, tensors are stored in CPU memory.

```
    x = torch.zeros(32, 32)
    assert x.device == torch.device("cpu")
```

However, what about GPUs?




```
    device = cuda_if_available()
```

In order to take advantage of the massive parallelism of GPUs, we need to move them to GPU memory.

```
    x = x.to(device)
```

Or create the tensor directly on the GPU:

```
    with torch.device(device):
        x = torch.zeros(32, 32)
        assert x.device == device
```

```
def tensor_einops():
    einops_motivation()
```

Einops is a library for manipulating tensors where dimensions are named.
It is inspired by Einstein summation notation (Einstein, 1916).
[Einops tutorial]

```
    einops_einsum()
    einops_reduce()
    einops_rearrange()
```

```
def einops_motivation():
```

Traditional PyTorch code:

```
    x = torch.ones(2, 2, 3)       # batch seq hidden
    y = torch.ones(2, 2, 3)       # batch seq hidden
    z = x @ y.transpose(-2, -1)  # batch seq seq
```

Easy to mess up the dimensions (what is -2, -1?)...

```
def einops_einsum():
```

Einsum is generalized matrix multiplication with good bookkeeping.

```
    x = torch.ones(3, 4)  # seq1 hidden
    y = torch.ones(4, 3)  # hidden seq2

    # Old way
    z = x @ y   # seq1 seq2
```

```
    # New (einops) way
    z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")

    Let's try a more complex example...

    x = torch.ones(2, 3, 4)  # batch seq1 hidden
    y = torch.ones(2, 3, 4)  # batch seq2 hidden

    # Old way
    z = x @ y.transpose(-2, -1)  # batch seq1 seq2

    # New (einops) way
    z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
    Dimensions that are not named in the output are summed over.

    # Or can use `...` to represent broadcasting over any number of dimensions
    z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")


def einops_reduce():
    You can reduce a single tensor via some operation (e.g., sum, mean, max, min).
    x = torch.ones(2, 3, 4)  # batch seq hidden

    # Old way
    y = x.sum(dim=-1)

    # New (einops) way
    y = reduce(x, "... hidden -> ...", "sum")


def einops_rearrange():
    Sometimes, a dimension represents two dimensions
    ...and you want to operate on one of them.

    x = torch.ones(3, 8)  # seq total_hidden
    ...where total_hidden is a flattened representation of heads * hidden1
    w = torch.ones(4, 4)  # hidden1 hidden2

    # Break up `total_hidden` into two dimensions (`heads` and `hidden1`
    x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)

    # Perform the transformation by `w`
    x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")

    # Combine `heads` and `hidden2` back together
    x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")


def tensor_operations_flops():
    Having gone through all the operations, let us examine their computational cost.

    A floating-point operation (FLOP) is a basic operation like addition (x + y) or multiplication (x y).

    Two terribly confusing acronyms (pronounced the same!):
    • FLOPs: floating-point operations (measure of computation done)
    • FLOP/s: floating-point operations per second (also written as FLOPS), which is used to measure the speed of
    hardware.
```

Intuitions

Training GPT-3 (2020) took 3.14e23 FLOPs. [article]
Training GPT-4 (2023) is speculated to take 2e25 FLOPs. [article]

H100 has a peak performance of 1979 teraFLOP/s with sparsity, 50% without [spec]

```
h100_flop_per_sec = 1979e12 / 2
```

8 H100s for 2 weeks:

```
total_flops = 8 * 2 * (60 * 60 * 24 * 7) * h100_flop_per_sec
```

Linear model

```
if torch.cuda.is_available():
    B = 16384  # Number of points
    D = 32768  # Dimension of each point
    K = 8192   # Number of outputs
else:
    B = 1024
    D = 256
    K = 64

x = torch.ones(B, D, device=cuda_if_available())
w = torch.randn(D, K, device=cuda_if_available())
y = x @ w
```

How many FLOPs is this matmul?
We have one multiplication (x[i][j] * w[j][k]) and one addition per (i, j, k) triple.

```
actual_num_flops = 2 * B * D * K
```

We can also time this operation to see how long it takes.

```
actual_time = benchmark(lambda: x @ w)
```

The actual FLOP/s of this operation:

```
actual_flop_per_sec = actual_num_flops / actual_time
```

Each GPU has a specification sheet that provides the peak performance.

- Example: [H100 spec]

Note that the FLOP/s depends heavily on the data type!

```
promised_flop_per_sec = get_promised_flop_per_sec(x.dtype)
```

Model FLOPs utilization (MFU)

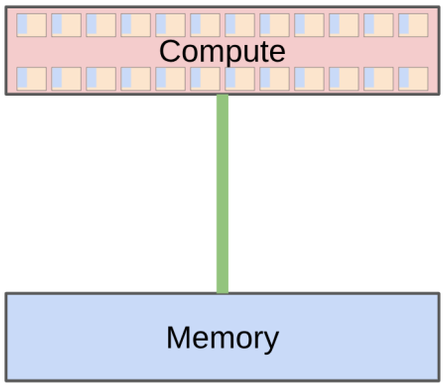
Definition: MFU = (actual FLOP/s) / (promised FLOP/s) [ignore communication/overhead]

```
mfu = actual_flop_per_sec / promised_flop_per_sec if promised_flop_per_sec else None
```

Usually, MFU of ≥ 0.5 is quite good!

But why is MFU not closer to 1?
To answer this question, we need to look more closely at how computations are done on GPUs...

```
def arithmetic_intensity():
```



How to compute a thing:

1. Send inputs from memory to accelerator
2. Perform computation
3. Send outputs from accelerator to memory

How long does this take?

Depends on two things:

1. Accelerator speed (FLOP/s)
2. Memory bandwidth (bytes/s)

```
assert h100_flop_per_sec == 1979e12 / 2  # Half without sparsity
assert h100_bytes_per_sec == 3.35e12

arithmetic_intensity_relu()
arithmetic_intensity_gelu()
arithmetic_intensity_dot_product()
arithmetic_intensity_matrix_vector_product()
arithmetic_intensity_matmul()

# Let's visualize it
roofline_plots()


def arithmetic_intensity_relu():
    n = 1024 * 1024
    x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
    y = torch.relu(x)

    bytes = (2 * n) + (2 * n)  # Read x, write y (bf16 is 2 bytes/float)
    flops = n  # n comparisons

    communication_time = bytes / h100_bytes_per_sec
    computation_time = flops / h100_flop_per_sec
```

Assume we can overlap communication and computation perfectly.

```
    total_time = max(communication_time, computation_time)
```

What is the bottleneck?

- Memory-bound: communication time > computation time
- Compute-bound: computation time > communication time

In this case, ReLU is memory-bound.

Alternative way to see this:

Accelerator intensity: how much work can the accelerator do per byte transferred?

```
    h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
```

Arithmetic intensity: how much actual work per byte for this workload?

```
arithmetic_intensity = flops / bytes  # ~1/4
```

What is the bottleneck?

- Memory-bound: arithmetic intensity < accelerator intensity
- Compute-bound: arithmetic intensity > accelerator intensity

```
assert arithmetic_intensity < h100_accelerator_intensity
```

In general, we'll find ourselves memory bound.
Can we increase arithmetic intensity?

```
def arithmetic_intensity_gelu():
    n = 1024 * 1024
    x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
    y = F.gelu(x)  # GELU(x) = 0.5 x (1 + tanh(sqrt(2/pi) (x + 0.044715 x^3)))

    bytes = (2 * n) + (2 * n)  # Read x, write y (bf16 is 2 bytes/float)
    flops = 20 * n  # tanh can be approximated in various ways (e.g., polynomials)

    arithmetic_intensity = flops / bytes

    h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
    assert arithmetic_intensity < h100_accelerator_intensity
```

Note that GeLU does more work than ReLU per byte moved, so it has higher arithmetic intensity.
But still memory-bound!
In other words, ReLU is not faster than GeLU (when doing things in an isolated way).

```
def arithmetic_intensity_dot_product():
    n = 1024 * 1024
    x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
    w = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
    y = x @ w

    bytes = (2 * n) + (2 * n) + 2  # Read x, read w, write y
    flops = 2 * n - 1  # n multiplications, n-1 additions

    arithmetic_intensity = flops / bytes  # ~1/2

    h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
    assert arithmetic_intensity < h100_accelerator_intensity
```

Memory-bound!

```
def arithmetic_intensity_matrix_vector_product():
    n = 1024
    x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
    w = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
    y = x @ w

    bytes = (2 * n) + (2 * n * n) + (2 * n)  # Read x, read w, write y
    flops = n * (2 * n - 1)  # n dot-products

    arithmetic_intensity = flops / bytes  # ~1
```

```
    h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
    assert arithmetic_intensity < h100_accelerator_intensity
```

Memory-bound!

```
def arithmetic_intensity_matmul():
    n = 1024
    x = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
    w = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
    y = x @ w

    bytes = (2 * n * n) + (2 * n * n) + (2 * n * n)  # Read x, read w, write y
    flops = n * n * (2 * n - 1)  # n^2 dot products

    arithmetic_intensity = flops / bytes  # ~n/3

    h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
    assert arithmetic_intensity > h100_accelerator_intensity
```

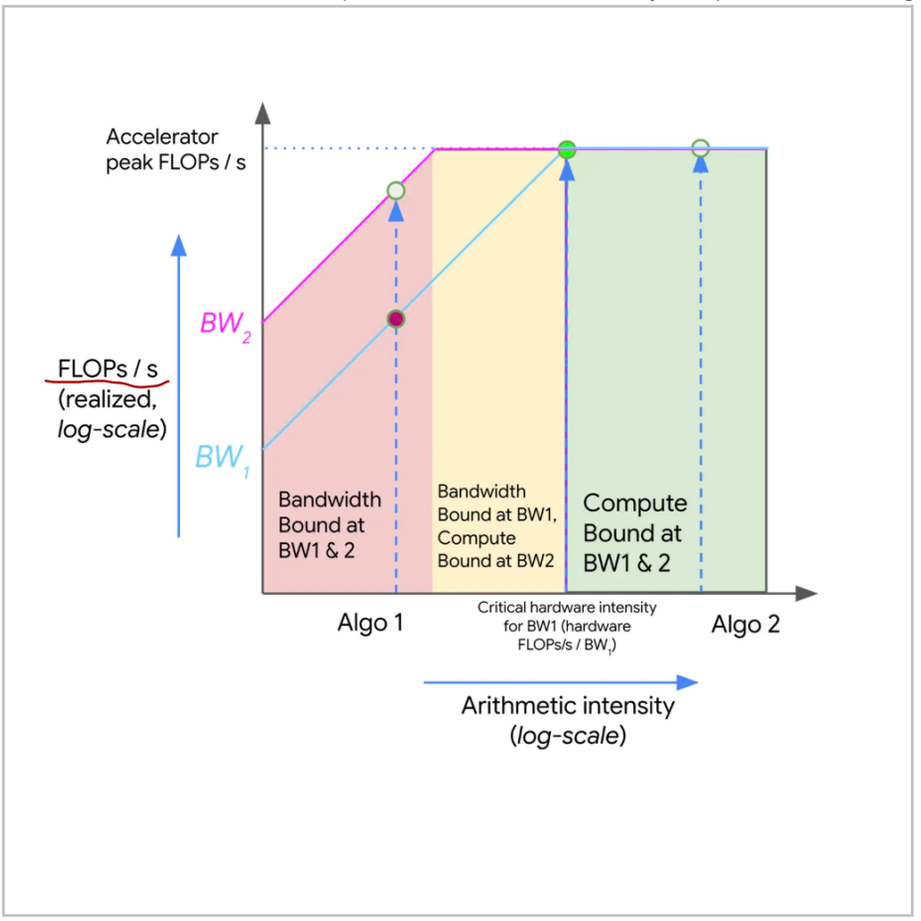
Finally, compute-bound!

As long as we have large matrices, we're compute-bound (saturating the accelerator).
Training Transformers involves big matrix multiplications.
Matrix-vector product is what happens during inference, which is why inference is memory-bound.

Note: arithmetic/accelerator intensity also depends on the precision (bf16 versus fp32).

```
def roofline_plots():
```

We can visualize the relationship between arithmetic intensity and performance using roofline plots.



- Each slice on the x-axis is a particular computation (with some arithmetic intensity)
- Each piecewise linear function corresponds to a particular hardware
- Kink is the accelerator intensity (transition from memory-bound to compute-bound)

We can now relate this back to MFU:

MFU = min(1, arithmetic-intensity / accelerator-intensity)

[reference]

```
def gradients_basics():
```

So far, we've constructed tensors and passed them through operations (forward).
Now, we're going to compute the gradient (backward).

As a simple example, let's consider the simple linear model:
y = 0.5 (x * w - 5)^2

Forward pass: compute loss

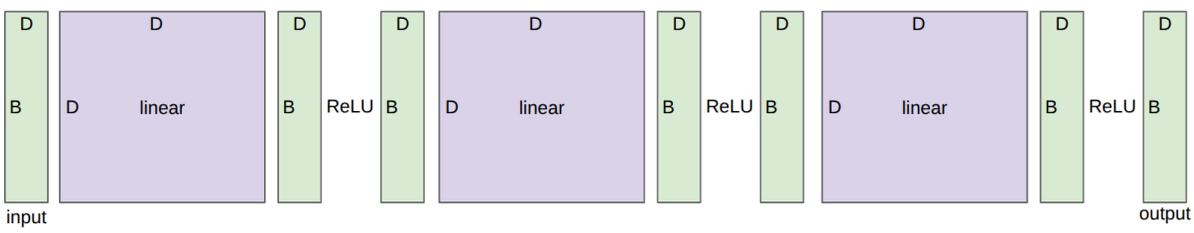
```
    x = torch.tensor([1., 2, 3])
    w = torch.tensor([1., 1, 1], requires_grad=True)  # Want gradient
    pred_y = x @ w
    loss = 0.5 * (pred_y - 5).pow(2)
```

Backward pass: compute gradients

```
    loss.backward()
    assert torch.equal(w.grad, torch.tensor([1, 2, 3]))
```

```
def gradients_flops():
```

Let us count the FLOPs for computing gradients.




```
    B = 1024  # Number of points
    D = 256   # Dimension
```

Define a simplified model (2-layer linear network):

```
    x = torch.ones(B, D, device=cuda_if_available())
    w1 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)
    w2 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)

    # Forward pass
    h1 = einsum(x, w1, "batch in, in out -> batch out")  # x
    h2 = einsum(h1, w2, "batch in, in out -> batch out")  # h1
    loss = (h2.mean() - 0)**2  # Regress everything to 0 (arbitrary)

    # Backward pass
    h1.retain_grad()  # For debugging
    h2.retain_grad()  # For debugging
    loss.backward()
```

Zoom in on one layer

Let's focus on the second layer (h2 = h1 @ w2)

Forward pass: Recall the number of forward FLOPs:

```
    num_forward_flops = 2 * B * D * D
```

Backward pass: How many FLOPs is running the backward pass?

We need to compute:

- h1.grad = d loss / d h1
- w2.grad = d loss / d w2

```
h1_grad = einsum(h2.grad, w2, "batch out, in out -> batch in")
assert torch.allclose(h1.grad, h1_grad)

w2_grad = einsum(h2.grad, h1, "batch out, batch in -> in out")
assert torch.allclose(w2.grad, w2_grad)

num_backward_flops = (2 * B * D * D) + (2 * B * D * D)
```

Note that the backward pass is 2x more expensive than the forward pass.

Consider all layers

This was just for w2, need to apply it to all parameters in the network.

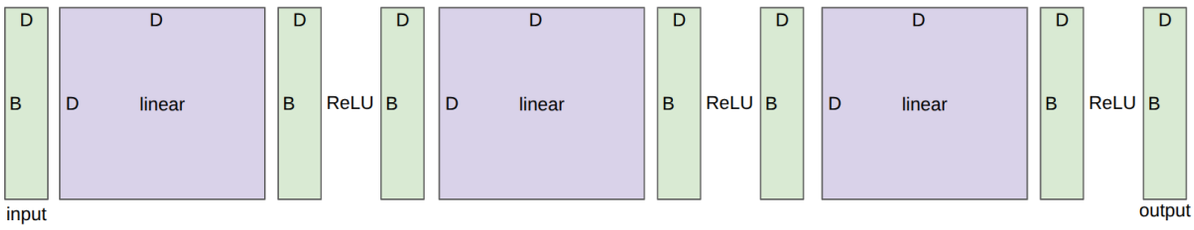
Putting it together:

- Forward pass: 2 (# data points) (# parameters) FLOPs
- Backward pass: 4 (# data points) (# parameters) FLOPs
- Total: 6 (# data points) (# parameters) FLOPs

This is for multilayer perceptrons (MLPs)

...but it turns out to be a good approximation for Transformers for short context lengths as well.

```
def deep_network():
```



Consider a deep network with L layers and D-dimensional inputs, activations, and outputs.

```
    # Define the network
    D = 8  # Dimensionality of input, activations, and output
    L = 3  # Number of layers
    model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())

    num_parameters = get_num_parameters(model)
    assert num_parameters == (D * D) * L

    # Run the model on a batch of data
    B = 4  # Batch size
    x = torch.randn(B, D, device=cuda_if_available())
    y = model(x)


class Block(nn.Module):
    """Simple block that applies a linear transformation followed by a ReLU nonlinearity."""
    def __init__(self, dim: int):
        super().__init__()
        self.weight = nn.Parameter(torch.randn(dim, dim) / math.sqrt(dim))
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x @ self.weight  # Linear
        x = F.relu(x)        # Activation
        return x


class DeepNetwork(nn.Module):
    """Map `dim`-vector to a `dim`-vector."""
    def __init__(self, dim: int, num_layers: int):
        super().__init__()
        self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Apply all the layers sequentially
        for layer in self.layers:
            x = layer(x)
        return x


def optimizer():
```

Recall our deep network.

```
    B = 2  # Batch size
    D = 4  # Dimensionality of input, activations, and output
    L = 3  # Number of layers
    model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
```

Let's define the AdaGrad optimizer

- momentum = SGD + exponential averaging of grad
- AdaGrad = SGD + averaging by grad^2
- RMSProp = AdaGrad but with exponential averaging of grad^2
- Adam = RMSProp + momentum

AdaGrad [Duchi+ 2011]

```
    optimizer = AdaGrad(model.parameters(), lr=0.01)
    state = model.state_dict()

    # Compute gradients
    x = torch.randn(B, D, device=cuda_if_available())
    y = torch.tensor([4., 5.], device=cuda_if_available())
    pred_y = model(x).mean()
    loss = F.mse_loss(input=pred_y, target=y)
    loss.backward()

    # Take a step
    optimizer.step()
    optimizer_state = {i: dict(p_state) for i, (p, p_state) in enumerate(optimizer.state.items())}

    # Free up the memory
    optimizer.zero_grad(set_to_none=True)
```

Memory

```
    num_parameters = D * D * L
    parameter_memory = 2 * num_parameters  # (2 bytes for bf16)
    gradient_memory = 2 * num_parameters  # (2 bytes for bf16)
    optimizer_state_memory = 4 * num_parameters  # (4 bytes for fp32)
    activation_memory = 2 * (B * D * L)  # (2 bytes for bf16)
```

It is customary to use fp32 for stability (accumulating averages over powers over many steps).

Optimizer state memory:

- AdaGrad: 4 bytes/parameter for storing second moments
- Adam: 8 bytes/parameter for storing first and second moments

```
# Putting it all together
total_memory = parameter_memory + activation_memory + gradient_memory + optimizer_state_memory
```

Compute (for one training step)

```
num_parameters = D * D * L
flops = 6 * B * num_parameters
```

Transformers

The accounting for a Transformer is more complicated, but the same idea.
Assignment 1 will ask you to do that.

Blog post describing memory usage for Transformer training [article]
Blog post describing FLOPs for a Transformer: [article]

```
class AdaGrad(torch.optim.Optimizer):
    def __init__(self, params: Iterable[nn.Parameter], lr: float = 0.01):
        super(AdaGrad, self).__init__(params, dict(lr=lr))

    def step(self):
        for group in self.param_groups:
            lr = group["lr"]
            for p in group["params"]:
                # Optimizer state
                state = self.state[p]
                grad = p.grad.data

                # Get squared gradients g2 = sum_{i<t} g_i^2
                g2 = state.get("g2", torch.zeros_like(grad))

                # Update optimizer state
                g2 += torch.square(grad)
                state["g2"] = g2

                # Update parameters
                p.data -= lr * grad / torch.sqrt(g2 + 1e-5)


def train_loop():
    # True linear function with weights (0, 1, 2, ..., D-1)
    D = 16  # Dimensionality
    true_w = torch.arange(D, dtype=torch.float32, device=cuda_if_available())

    # Data loader that generates (x, y) pairs
    B = 4  # Batch size
    def get_batch() -> tuple[torch.Tensor, torch.Tensor]:
        x = torch.randn(B, D).to(cuda_if_available())
        true_y = x @ true_w
        return (x, true_y)

    # Define the model and optimizer
    L = 2  # Number of layers
    model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
```

```
    optimizer = AdaGrad(model.parameters(), lr=0.01)

    # Train!
    num_train_steps = 10
    for t in range(num_train_steps):
        # Get data
        x, y = get_batch()

        # Forward (compute loss)
        pred_y = model(x).mean()
        loss = F.mse_loss(pred_y, y)

        # Backward (compute gradients)
        loss.backward()

        # Update parameters
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)


def gradient_accumulation():
```

Large batch sizes: improve training stability

However, activation memory scales with batch size, so might run out.

```
    B = 64      # Batch size
    D = 1024    # Dimensionality
    L = 16      # Number of layers
    activation_memory = 2 * B * D * L  # (2 bytes for bf16)
```

Gradient accumulation:

- Compute gradient on micro batches
- Accumulate the gradients (don't zero it out)
- Every batch_size / micro_batch_size steps, update the parameters and zero out the gradients

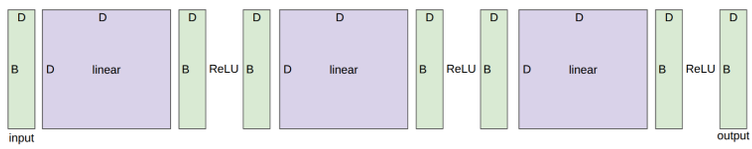
```
    micro_batch_size = 256
    activation_memory = 2 * micro_batch_size * D * L  # (2 bytes for bf16)
```

激活检查点（重算）

```
def activation_checkpointing():
```

For training, we need to store the activations of all layers

For inference, we don't compute gradients, so we only need to store the current layer's activations.



The memory usage is

```
    B = 64      # Batch size
    D = 1024    # Dimensionality
    L = 16      # Number of layers

    x = torch.randn(B, D, device=cuda_if_available(), requires_grad=True)
    activation_memory = 2 * B * D * L

    model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
    memory = get_max_memory_usage(lambda: model(x).sum().backward())
```

Can we reduce this?

Activation checkpointing = gradient checkpointing = rematerialization

Key idea:

- Forward pass: keep only activations at subset of layers
- Backward pass: recompute the missing activations from the last checkpoint

Philosophy: tradeoff memory for compute

```
# Store all activations:    x g1 h1 g2 h2 g3 h3 g4 h4
# Activation checkpointing: x    h1    h2    h3    h4
```

```
# Define the model with checkpointing
model = DeepNetworkCheckpointed(dim=D, num_layers=L).to(cuda_if_available())
checkpointed_memory = get_max_memory_usage(lambda: model(x).sum().backward())
```

Can we reduce this even more, especially for deep networks (large L)?

```
# Store all layers:   | h1 h2 h3 h4 h5 h6 h7 h8 h9 |
# Store no layers:    |                            |
# Store some layers:  |    h3       h6          h9 |
```

How frequently to checkpoint?

- If store each layer's activations, then activation memory is O(L) and no recomputation.
- If store no activations, then activation memory is O(1) and compute is O(L^2) (recompute from the start for each layer).
- If store every sqrt(L) layers, then activation memory is O(sqrt(L)) and O(L) recomputation.

```
class DeepNetworkCheckpointed(nn.Module):
    """Same as DeepNetwork, but with activation checkpointing."""
    def __init__(self, dim: int, num_layers: int):
        super().__init__()
        self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Apply all the layers sequentially
        for layer in self.layers:
            # KEY: only store activations at checkpoints, recompute the rest
            x = torch.utils.checkpoint.checkpoint(layer, x)
        return x

############################################################

def get_memory_usage(x: torch.Tensor):
    return x.numel() * x.element_size()


def get_promised_flop_per_sec(dtype: torch.dtype) -> float:
    """Return the peak FLOP/s for `device` operating on `dtype`."""
    if not torch.cuda.is_available():
        # No CUDA device available, so can't get FLOP/s
        return 1
    properties = torch.cuda.get_device_properties(cuda_if_available())

    if "A100" in properties.name:
        # https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf
        if dtype == torch.float32:
            return 19.5e12
        if dtype in (torch.bfloat16, torch.float16):
            return 312e12
        raise ValueError(f"Unknown dtype: {dtype}")
```

```
809
810     if "H100" in properties.name:
811         # https://www.nvidia.com/en-us/data-center/h100/
812         if dtype == torch.float32:
813             return 67.5e12
814         if dtype in (torch.bfloat16, torch.float16):
815             return 1979e12 / 2  # 1979 is for sparse, dense is half of that
816         raise ValueError(f"Unknown dtype: {dtype}")
817
818     if "B200" in properties.name:
819         # https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-
datasheet.pdf
820         if dtype == torch.float32:
821             return 75e12
822         if dtype in (torch.bfloat16, torch.float16):
823             return 4.5e15 / 2  # 4.5e15 is for sparse, dense is half of that
824         raise ValueError(f"Unknown dtype: {dtype}")
825
826     # Unknown GPU: return None so caller can handle gracefully
827     return None
828
829
830 def benchmark(func, num_trials: int = 5) -> float:
831     """Return the number of seconds required to perform `func`."""
832
833     # Wait until previous CUDA threads are done
834     if torch.cuda.is_available():
835         torch.cuda.synchronize()
836
837     def run():
838         # Perform the operation
839         func()
840
841         # Wait until CUDA threads are done
842         if torch.cuda.is_available():
843             torch.cuda.synchronize()
844
845     # Time the operation `num_trials` times
846     total_time = timeit.timeit(run, number=num_trials)
847
848     return total_time / num_trials
849
850
851 def get_num_parameters(model: nn.Module) -> int:
852     return sum(param.numel() for param in model.parameters())
853
854
855 if __name__ == "__main__":
856     main()
```